

MSc Quantum Science and Technology

Master Thesis

Toward ML-Based State Discrimination on FPGA in Open-Source Quantum Control Electronics

Author: Ender Jose Barillas Rodriguez

Supervised by: Joel Pérez Díaz

NIUB: 23254663
Faculty of Physics, University of Barcelona
08028, Barcelona
July 2026

Acknowledgements

I would like to thank the team at Qilimanjaro Quantum Tech for their guidance and support throughout this project. Special thanks to my supervisor, Joel Pérez Díaz, for his patience and direction, to David A. for carefully reviewing this manuscript, and to Fabio H. and the rest of the team for their help in getting the board up and running. I am also grateful to Bruno and the MQST programme for the chance to take a shot at joining a new field, and to my classmates for making the year what it was. On a personal note, I thank Arianna for helping me survive in Barcelona, and my parents for their constant support. This thesis was carried out during an internship at Qilimanjaro from February to July 2026.

Abstract

Low-latency and high-fidelity qubit state measurement is essential for superconducting quantum computing, particularly for enabling mid-circuit measurements and real-time feedback. While commercial control hardware increasingly supports on-FPGA state discrimination, these platforms operate on closed firmware that precludes custom signal processing pipelines. Open-source FPGA frameworks offer an alternative path: full programmatic access to the readout chain, letting researchers integrate custom algorithms, including machine learning, directly into the measurement workflow.

This thesis documents the setup and characterisation of a QICK-based [1] readout system on a Xilinx ZCU216 RFSoC platform, covering FPGA configuration, remote operation, DDS-based waveform generation, and analogue front-end characterisation. The work establishes a functional signal chain capable of driving and reading out superconducting resonators, and examines the hardware constraints relevant to dispersive qubit readout, including balun response and filter rolloff across the relevant frequency bands.

Using existing single-shot readout data from a superconducting qubit device, we design and evaluate a machine-learning-assisted state discrimination pipeline. On integrated IQ data, linear discriminant analysis is already near-optimal: none of the twenty MLP architectures we tested improves on it, as expected from the Gaussian structure of the IQ clouds. The only meaningful gain comes from a lightweight 1D CNN trained on physically-grounded synthetic raw ADC traces, which recovers shots corrupted by mid-readout T_1 relaxation that integrated methods cannot distinguish. On a synthetic test set the CNN reaches 99.75% overall fidelity and 98.4% accuracy on relaxation shots, against 44.3% for LDA on the same 122 shots. This model is trained and evaluated entirely offline; real-time deployment within the QICK firmware was not achieved in this work. The thesis instead closes by outlining the path toward it: the data collection procedure required on the ZCU216, the model adaptation and quantisation constraints imposed by the programmable logic fabric, and the methodology needed to test whether deployment improves classification speed, fidelity, or both.

Keywords: superconducting qubits, readout, FPGA, QICK, RFSoC, dispersive measurement, mid-circuit measurement, machine learning, state discrimination

Contents

1 Introduction

Quantum computing holds the promise of solving classically intractable problems in optimisation, simulation, and cryptography. Among the leading hardware platforms, superconducting qubits stand out for their relatively fast gate times, high coherence, and compatibility with existing microwave engineering infrastructure [2]. A central challenge that must be overcome on the road to fault-tolerant operation is *qubit state measurement*: reading out the quantum information stored in a qubit quickly, accurately, and with minimal back-action on unmeasured qubits.

1.1 Motivation

Classical error correction requires measurements that are not only high-fidelity but also *fast* relative to qubit coherence times. Crucially for quantum error correction, it also requires *mid-circuit measurements*: the ability to measure ancilla qubits repeatedly during a quantum algorithm, use the results to extract error syndromes in real time, and continue the computation on the data qubits. This demands a control electronics stack that can make a state discrimination decision on-chip in microseconds, the timescale of the readout integration window itself, rather than the milliseconds typical of a software round-trip to a host CPU.

Field-Programmable Gate Arrays (FPGAs) offer a compelling solution. Their deterministic, sub-microsecond timing enables real-time signal processing close to the qubit hardware. Commercial control platforms such as Qblox, Zurich Instruments, and Quantum Machines already exploit this, providing on-FPGA state discrimination within closed, proprietary firmware stacks. However, closed firmware precludes custom signal processing pipelines: researchers cannot inject bespoke algorithms, machine-learning classifiers among them, into the real-time decision path. The Quantum Instrumentation Control Kit (QICK) [1] addresses this gap as an open-source FPGA firmware and software framework that turns Xilinx Radio-Frequency System-on-Chip (RFSoC) boards into complete qubit controllers, combining fast digital-to-analogue converters (DACs), analogue-to-digital converters (ADCs), and a tightly integrated soft-processor into a single compact platform. Full access to the programmable logic fabric makes it possible to define the entire readout and discrimination pipeline, including the integration of custom ML models, in a way that closed commercial firmware does not permit.

1.2 Goals and Contributions

Despite the availability of QICK, assembling a *general-purpose* readout pipeline from an uncharacterised board to a working single-shot state discriminator requires integrating many layers: physical wiring, FPGA configuration, phase calibration, signal optimisation, and machine-learning post-processing. The lack of a unified reference implementation makes this process time-consuming and error-prone, especially for multi-qubit scenarios.

This thesis makes the following contributions:

1. **Full-stack integration** of the QICK framework on a Xilinx ZCU216 RFSoC board with an XM655 analogue front-end module, including remote operation via a persistent Pyro4 server.

2. **Phase-coherent readout calibration**: a systematic procedure for measuring and compensating the random DAC–ADC phase offset that arises at every board power-on, enabling reproducible IQ measurements across sessions.

3. **Readout data-path design**: a characterisation of the trade-offs between decimated and accumulated readout modes and a complete resonator spectroscopy workflow suitable for both single-qubit and multiplexed measurements.

4. **Real-time feedback primitives**: demonstration of the `condj` conditional-branch primitive on the tProcessor in loopback, and design of an active qubit reset sequence combining dispersive readout, thresholding, and a calibrated π pulse. This establishes the infrastructure needed for mid-circuit measurement once the board is connected to a qubit chip.

5. **ML-assisted state discrimination**: offline design and evaluation of a lightweight ML discrimination pipeline on existing experimental SSRO data, establishing that LDA is near-optimal for integrated IQ inputs, and demonstrating via synthetic raw traces that a 1D CNN recovers T_1-relaxation shots irrecoverable by integration-based methods. A concrete path toward FPGA deployment is outlined.

1.3 Thesis Outline

Section 2 reviews the relevant theory of superconducting qubits, circuit quantum electrodynamics (cQED), and dispersive readout. Section 3 describes the hardware platform. Section 4 covers system integration and calibration. Section 5 presents the readout architecture and measurement workflows. Section 6 describes real-time feedback and mid-circuit measurement. Section 7 evaluates machine-learning-based state discrimination. Section 8 summarises the results and outlines future directions.

2 Theoretical Background

2.1 Superconducting Qubits

Superconducting qubits are anharmonic LC oscillators whose nonlinearity is provided by one or more Josephson junctions [2]. The Josephson junction acts as a nonlinear inductor with energy $E_J \cos\hat{\varphi}$, where $\hat{\varphi}$ is the superconducting phase across the junction and E_J is the Josephson energy. Combined with a shunt capacitance C (charging energy $E_C = e^2/2C$), the system forms an artificial atom with a discrete, anharmonic spectrum that can be addressed at the transition frequency ω_{01}. Several qubit designs have been developed that exploit this anharmonicity in different parameter regimes; this thesis is primarily concerned with two: the transmon and the fluxonium.

Transmon. The transmon [3] operates deep in the regime $E_J \gg E_C$, making its energy levels insensitive to charge noise. Its transition frequency lies typically in the 4 GHz to 8 GHz range, and its anharmonicity $\alpha = \omega_{12} - \omega_{01}$ is of order $-E_C/\hbar$, typically a few hundred MHz.

Fluxonium. The fluxonium [4] replaces the transmon's large capacitor with a superinductance L (inductive energy $E_L = (\Phi_0/2\pi)^2/L$, where Φ_0 is the flux quantum), threading the loop with an external flux Φ_z. The external flux tunes the resulting potential: near $\Phi_z = 0$ the low-lying states are plasmon-like, while near $\Phi_z = \Phi_0/2$ a double well forms and the lowest states localise in the two wells. Fluxonium qubits offer long coherence times, and their readout landscape is richer than the transmon's because the dispersive shift χ varies strongly with flux bias. Near the half-flux point the two lowest states localise in

opposite wells and carry persistent currents of opposite sign, giving a large, flux-tunable state-dependent shift that can be exploited for high-contrast readout [4].

2.2 Circuit Quantum Electrodynamics

In the circuit QED (cQED) architecture [5], a qubit is coupled dispersively to a microwave resonator. The resonator acts both as a readout bus and as a filter that protects the qubit from radiative decay into the measurement chain. The qubit–resonator Hamiltonian in the dipole-coupling (Rabi) form (setting $\hbar = 1$) is

$$\hat{H} = \hat{H}_{\text{qubit}} + \omega_r \hat{a}^\dagger \hat{a} + g\, \hat{n}_q (\hat{a} + \hat{a}^\dagger), \tag{1}$$

where ω_r is the resonator frequency, $\hat{a}$ ($\hat{a}^\dagger$) is the resonator annihilation (creation) operator, g is the qubit–resonator coupling strength, and $\hat{n}_q$ is the relevant qubit operator (charge for capacitive coupling) [2]. When the qubit–resonator detuning is large compared to the coupling ($|g| \ll |\omega_{ij} - \omega_r|$), second-order perturbation theory in g yields an effective qubit-state-dependent shift of the resonator frequency, derived in Section 2.3 below.

2.3 Dispersive Readout

When $|g_{ij}| \ll |\omega_{ij} - \omega_r|$ for all relevant qubit transitions $i \leftrightarrow j$, second-order perturbation theory in g yields a qubit-state-dependent shift of the resonator frequency [6]:

$$\hat{H}_{\text{disp}} = \left(\omega_r + \sum_i \chi_i \, |i\rangle \langle i| \right) \hat{a}^\dagger \hat{a}, \quad \chi_i = \sum_{j \neq i} \frac{|g_{ij}|^2}{\omega_{ij} - \omega_r} - \frac{|g_{ji}|^2}{\omega_{ji} - \omega_r}, \tag{2}$$

where $g_{ij} = \langle i | g\, \hat{n}_q | j \rangle$ and $\omega_{ij} = \omega_j - \omega_i$. The textbook two-level dispersive shift $\chi = \chi_1 - \chi_0$ follows from truncating to the computational subspace, but for fluxonium-class qubits the *straddling regime*, in which the resonator frequency falls between the $0 \to 1$ and $1 \to 2$ transition frequencies, makes this truncation unsafe: the dispersive shifts from the $0 \to 1$ and $1 \to 2$ transitions then carry opposite signs and partially cancel, so higher levels must be included [3].

In practice, one probes the resonator at its bare frequency ω_r and observes a state-dependent phase shift of the transmitted or reflected signal. For a single-qubit system, the resonator sits at $\omega_r + \chi_0$ when the qubit is in $|0\rangle$ and at $\omega_r + \chi_1$ when in $|1\rangle$, resulting in an IQ-plane separation that grows with the dimensionless cavity pull $2|\chi|/\kappa$, where κ is the resonator linewidth [5, 7].

Fluxonium dispersive readout. Because the fluxonium spectrum varies strongly with the external flux Φ_z, so does its dispersive shift $\chi = \chi_1 - \chi_0$: the state-dependent coupling to the resonator, and hence the readout contrast, can be tuned through the flux bias. This tunability is a key motivation for the fluxonium readout optimisation work being pursued in related projects within the Qilimanjaro team.

2.4 Direct Digital Synthesis

Direct Digital Synthesis (DDS) generates analogue waveforms by accumulating a phase register at a fixed clock rate and mapping the instantaneous phase value to a digital amplitude via a look-up table. A DDS numerically controlled oscillator (NCO) produces a tone at frequency

$$f_{\text{out}} = \frac{f_{\text{clk}} \cdot \Delta\phi}{2^N}, \tag{3}$$

where f_{clk} is the system clock, $\Delta\phi$ is the phase increment per clock cycle, and N is the bit width of the phase accumulator. DDS avoids the spur and image problems associated with analogue IQ mixers because all imperfections are digital and predictable. In QICK, the tProcessor drives DDS engines on both the DAC (transmit) and ADC (receive) sides, enabling phase-locked signal generation and coherent demodulation [1].

2.5 State Discrimination

Following dispersive readout, the measured IQ quadrature amplitudes (I, Q) form two clusters in the complex plane, one for $|0\rangle$ and one for $|1\rangle$. Single-shot state discrimination assigns each measurement to one of these clusters. The simplest approach is a linear threshold: a hyperplane in the IQ plane that minimises the total misclassification probability, which for Gaussian clusters of equal covariance is the perpendicular bisector of the line connecting the centroids [8]. The single-shot readout fidelity is

$$\mathcal{F} = 1 - \frac{1}{2}\left(p(1|0) + p(0|1)\right), \tag{4}$$

where $p(1|0)$ is the probability of assigning $|1\rangle$ given that the qubit was prepared in $|0\rangle$, and vice versa. More sophisticated classifiers, such as quadratic discriminant analysis or neural networks, can improve fidelity when the clusters are non-Gaussian or when multi-level leakage is present.

2.6 Machine Learning for State Discrimination

The state discrimination problem defined in Section 2.5 is a binary classification task: given a measurement outcome $\mathbf{x} \in \mathbb{R}^d$, assign it to one of two classes ($|0\rangle$ or $|1\rangle$). The choice of classifier and its optimality depend on the statistical structure of the data.

Linear Discriminant Analysis. If the two classes are distributed as multivariate Gaussians with equal covariance, $p(\mathbf{x} \mid s) = \mathcal{N}(\boldsymbol{\mu}_s, \boldsymbol{\Sigma})$ for $s \in \{0, 1\}$, then the Bayes-optimal decision boundary is the hyperplane

$$(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)^\top \boldsymbol{\Sigma}^{-1} \left(\mathbf{x} - \tfrac{1}{2}(\boldsymbol{\mu}_0 + \boldsymbol{\mu}_1)\right) = 0, \tag{5}$$

which is exactly the decision boundary found by Linear Discriminant Analysis (LDA) [8]. Because this boundary is already optimal under the Gaussian equal-covariance assumption, no more expressive classifier, multilayer perceptrons (MLPs) included, can improve on it: an MLP can at best approximate this boundary, never exceed the Bayes optimum. When the covariances differ between classes, Quadratic Discriminant Analysis (QDA) uses class-specific covariances $\boldsymbol{\Sigma}_0, \boldsymbol{\Sigma}_1$ and yields a curved boundary that is Bayes-optimal under the unequal-covariance Gaussian model.

Limitations of integrated IQ input. Both LDA and QDA operate on a single integrated (I, Q) pair per shot: the time-averaged quadrature amplitudes over the readout window. This compression discards all temporal information. In particular, if the qubit relaxes from $|1\rangle$ to $|0\rangle$ at some time t^* within the readout window, the integrator averages over both signal levels and produces a point that falls between the two clusters. No classifier operating on the integrated input can recover these shots, regardless of its expressive power, because the information about the true prepared state has been destroyed by the integration.

Convolutional Neural Networks on raw traces. A one-dimensional convolutional neural network (1D CNN) takes the full raw ADC time trace as input, preserving the temporal structure that integration discards. A CNN applies a bank of learned filters across the time axis via the convolution operation

$$(f * x)[t] = \sum_{\tau=0}^{k-1} f[\tau]\, x[t-\tau], \tag{6}$$

where f is a learned filter of length k and x is the input signal. By stacking convolutional layers, the network can detect local temporal features at increasing scales, such as the step discontinuity in the IQ trajectory that occurs when the qubit relaxes mid-shot. This makes CNNs well-suited to recovering T_1-relaxation shots that are irrecoverable from integrated IQ, at the cost of requiring raw trace data rather than accumulated IQ pairs.

3 Hardware Platform

3.1 Xilinx ZCU216 RFSoC

The Xilinx Zynq UltraScale+ ZCU216 evaluation board [9] is built around an RFSoC device that integrates, on a single chip, a quad-core ARM Cortex-A53 processor, an FPGA fabric, and 16 RF-DAC and 16 RF-ADC channels with sampling rates up to 9.85 GS/s and 2.5 GS/s, respectively. At 14-bit resolution, the converters can directly synthesise and capture signals in the 0 GHz to 6 GHz range without an external frequency up-conversion stage, provided the analogue signal chain is suitably conditioned.

The board runs the PYNQ Linux distribution, which exposes the FPGA fabric and the converters through Python libraries, making it straightforward to implement and iterate on firmware/software stacks. Table 1 summarises the key specifications relevant to this thesis.

Table 1: Selected ZCU216 specifications relevant to qubit readout.

Parameter	Value	Note
DAC channels	16	Differential GSPS
DAC rate	up to 9.85 GS/s	Per channel
DAC resolution	14 bit	
ADC channels	16	Differential GSPS
ADC rate	up to 2.5 GS/s	Per channel
ADC resolution	14 bit	
FPGA fabric	UltraScale+	930k LUT
Processor	4× Cortex-A53	1.5 GHz
Operating system	PYNQ Linux	

3.2 QICK Software and Firmware

The Quantum Instrumentation Control Kit (QICK) [1, 10] is an open-source framework originally developed at Fermilab that turns an RFSoC board into a self-contained qubit controller. It consists of two layers:

Firmware. A set of Vivado IP blocks synthesised and placed onto the FPGA fabric, comprising:

- **Signal generators**: DDS-based waveform engines that drive the DACs with arbitrary envelopes, constant tones, or flat-top pulses. Generator 6 (physical port 2_231 on the ZCU216) is used as the primary readout generator throughout this work.
- **Readout blocks**: ADC-side DDS demodulators that down-convert the received signal to baseband. Each readout block provides both a decimated output (time-domain I/Q waveform) and an accumulation buffer (averaged I/Q pair). Readout channel 0 (ADC port 0_226) corresponds to the first ADC pair on the XM655 breakout.
- **tProcessor V2**: a soft-processor that orchestrates pulse timing with nanosecond precision. It executes a custom instruction set including direct memory writes (`memri`), register arithmetic (`mathi`, `regwi`), conditional branches (`condj`), and pulse triggers. The ZCU216 uses the 64-bit variant of the tProcessor, which extends the original 32-bit design with wider arithmetic and richer branching.

Software. A Python library (the `qick` package) that runs on the ARM core and exposes high-level classes for writing and running experiments:

- `QickSoc`: instantiates the full hardware stack, loads the bitstream, and provides access to the converters and tProcessor.
- `QickProgram` / `AveragerProgram` / `RAveragerProgram`: abstractions for writing measurement sequences at increasing levels of automation. `RAveragerProgram` enables FPGA-internal parameter sweeps via the tProcessor's `update()` hook, avoiding Python overhead between experiment points.
- Remote operation: a persistent Pyro4 server running on the board makes it possible to control the board from a laptop over a standard network connection, without reinitialising the FPGA between notebook sessions. This design also preserves the phase calibration across reconnects.

3.3 XM655 Analogue Front-End Module

The ZCU216's RF converters use differential signalling on High-Density (HD) headers (JHC3 and JHC5 on the board). The XM655 module is Xilinx's daughter-card solution: it provides BALUN (balanced-to-unbalanced) transformers that convert the differential DAC outputs and ADC inputs to/from single-ended SMA ports, along with optional filtering.

The key constraint introduced by the XM655 in this work is its bandwidth: the BALUNs cover 10 MHz to 6000 MHz, so resonator frequencies above 6 GHz require connections directly on the HD header pins using pigtail cables.

The loopback path used for calibration experiments consists of: DAC port 2_231 (JHC3) → HD2-SMA pigtail → BALUN → SMA cable → BALUN → HD2-SMA pigtail → ADC port 0_226 (JHC5).

Figure 1 shows the high-level signal flow from the tProcessor to the qubit chip and back.

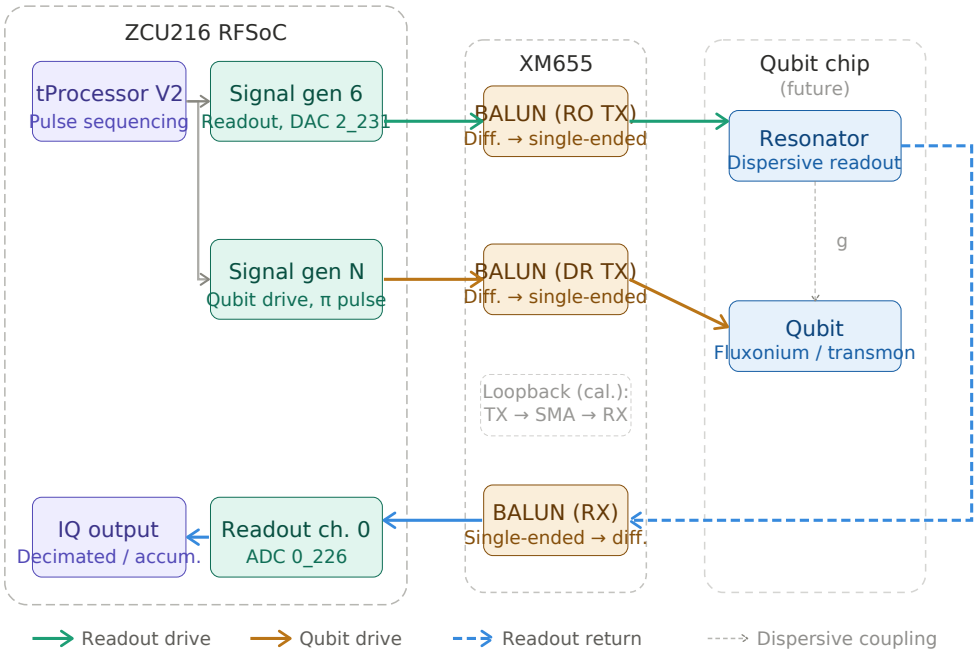

Figure 1: System signal chain for the QICK-based readout platform. The tProcessor V2 sequences pulses on two generator channels: signal gen 6 (DAC 2_231) drives the readout tone through the XM655 TX BALUN to the resonator; signal gen N drives the qubit directly via a second TX BALUN channel for π-pulse control (the specific channel number depends on the final wiring and is left as N here). The reflected readout tone returns through the XM655 RX BALUN to ADC 0_226 and is demodulated to an IQ pair by readout channel 0. The XM655 BALUNs cover $10\,\mathrm{MHz}$ to $6000\,\mathrm{MHz}$; resonator frequencies above $6\,\mathrm{GHz}$ require direct HD-header pigtail connections, bypassing the XM655. The qubit chip and its connections are shown as a design target; the board was not connected to a qubit chip within the scope of this work. Calibration experiments use the loopback path (TX BALUN $\rightarrow$ SMA cable $\rightarrow$ RX BALUN) shown inside the XM655 container.

4 System Integration

4.1 FPGA Configuration and Board Bring-up

The ZCU216 boots the PYNQ Linux image from an SD card. After boot, the QICK bitstream is loaded onto the FPGA fabric by instantiating a `QickSoc` object in Python, which calls the PYNQ overlay loader and configures all IP blocks. Because the RFSoC converters are controlled through AXI register maps rather than external chip-select lines, initialisation is entirely software-driven.

The board is accessed over SSH. To avoid re-initialising the hardware for every new notebook session, a persistent Pyro4 object server is launched once on the board and kept running:

```
# On the board (runs once per power cycle)
import Pyro4, qick
soc = qick.QickSoc()
daemon = Pyro4.Daemon()
uri = daemon.register(soc, "qick.soc")
Pyro4.locateNS().register("qick.soc", uri)
daemon.requestLoop()
```

From a remote laptop:

```python
import Pyro4
soc = Pyro4.Proxy("PYRONAME:qick.soc")
soccfg = soc  # lightweight config object
```

This design preserves the phase calibration state (see Section 4.3) across reconnects and makes experiment iteration fast.

4.2 Channel Mapping

The QICK firmware assigns each physical DAC output to a numbered *generator channel* and each physical ADC input to a numbered *readout channel*. On the ZCU216 with the default QICK bitstream, the mapping relevant to this work is shown in Table 2.

Table 2: QICK channel mapping on the ZCU216.

QICK Channel	Physical Port	HD Header	Function
Generator 6	DAC 2_231	JHC3	Readout / control drive
Readout 0	ADC 0_226	JHC5	IQ acquisition

4.3 Phase Calibration

Each time the ZCU216 is powered on or the FPGA bitstream is reloaded, the DDS oscillators in the DAC and ADC blocks boot with a random mutual phase offset ϕ. Without compensation, IQ measurements rotate unpredictably between sessions, making it impossible to compare results across power cycles.

Phase model. A loopback tone from DAC 2_231 to ADC 0_226 measures ϕ at a given frequency. Because the two oscillators are effectively separated by a fixed time delay τ (the total group delay around the loopback path), the phase offset scales linearly with frequency:

$$\phi(f) = \phi_0 - 360^\circ \cdot \tau \cdot f, \tag{7}$$

where ϕ_0 is a constant offset and τ is extracted from the fit. The fitted τ reflects the combined pipeline delay of the DAC and ADC signal chains, including the digital up- and down-conversion stages (DUC/DDC) and internal FIFOs, rather than the physical cable propagation delay, which for a bench loopback of 1–2 m contributes only a few nanoseconds.

Calibration procedure. The phase calibration sweeps a range of frequencies f_i near the target readout frequency, records the averaged IQ phasor at each point, and extracts $\phi_i = \arg\langle I + jQ \rangle$. The pair (ϕ_0, τ) is then fitted using `scipy.optimize.least_squares`, taking care to handle the $0^\circ/360^\circ$ phase wrap correctly. The fit is considered reliable when the signal-to-noise ratio satisfies $|\text{mag}| \gg \text{RMS}$ (e.g. $637 \gg 1.8$ ADU in a typical calibration run).

The resulting `res_phase` value:

$$\phi_{\text{res}} = \phi_0 - 360^\circ \cdot \tau \cdot f_{\text{res}} \tag{8}$$

is stored in the experiment configuration dictionary and passed to the readout pulse via `soccfg.deg2reg(res_phase)`, which converts degrees to the integer register format expected by the tProcessor. This calibration must be re-run after every board reboot or bitstream reload, but remains valid throughout a measurement session.

Measurement cadence. A summary of the recommended start-of-session checklist is:

1. Power on and wait for PYNQ to boot (≈60 s).
2. Connect the loopback (DAC 2_231 → BALUN → SMA → BALUN → ADC 0_226) and run the phase calibration notebook.
3. Identify the correct `adc_trig_offset` using decimated readout (see Section 5.2).
4. Switch to accumulated readout for all data-taking.
5. At session end, call `soc.reset_gens()` to halt all running generators.

5 Readout Architecture

5.1 DDS-Based Signal Generation

In a standard heterodyne readout chain, a local oscillator (LO) at frequency f_{LO} is mixed with a baseband envelope to up-convert the readout tone to the resonator frequency f_r. This analogue mixing process introduces image products and LO leakage that degrade fidelity, requiring careful IQ imbalance correction.

QICK avoids analogue mixing entirely. Both the readout drive and the demodulation reference are produced by the on-chip DDS engines, which operate at the direct output frequency of the RFSoC converters (up to several GHz). The waveform generated for qubit readout has the form

$$s(t) = A \cdot \mathrm{env}(t) \cdot \cos(2\pi f_r t + \phi_{\mathrm{res}}), \tag{9}$$

where $\mathrm{env}(t)$ is the pulse envelope (constant, Gaussian, DRAG, etc.), f_r is programmed directly into the DDS frequency register, and ϕ_{res} is the phase correction from calibration. The only noise sources are digital: quantisation, clock jitter, and DDS phase truncation spurs, all of which are deterministic and can be mitigated by appropriate filtering and sample-rate choice.

5.2 Decimated vs. Accumulated Readout

After the ADC samples the incoming signal, the QICK readout block demodulates it by multiplying with the NCO reference tone and low-pass filtering. Two output modes are available:

Decimated readout. Stores the full time-domain I/Q waveform over the readout window (e.g. 1000 samples). This is essential for setup: it reveals when the readout pulse arrives relative to the trigger (`adc_trig_offset`), whether the integration window is correctly positioned, and whether the signal shape is as expected. The per-sample noise on the decimated output is ∼1.9 ADU RMS; this is the raw sample-level noise floor, distinct from the per-shot integrated IQ noise reported for accumulated readout below.

Accumulated readout. The FPGA sums all ADC samples within the readout window into a single (I, Q) pair in hardware, one pair per shot. This is the standard mode for SSRO: each shot yields one IQ point, and the discrimination classifier operates on that point. The per-shot integrated IQ noise is determined by the within-window averaging over T_{win} samples, not by the number of repeated shots. From the real SSRO data, the per-shot cluster widths are approximately 0.65–0.97 ADU per quadrature (state-dependent), with

an IQ separation of 2.53 ADU and a projected SNR of 2.46, yielding a single-shot fidelity of 0.906 [11].

For applications that do *not* require single-shot resolution, such as resonator spectroscopy or Rabi calibration, the same shot can be repeated N_{reps} times and the results averaged, reducing the noise on the *mean* phasor as $1/\sqrt{N_{\text{reps}}}$:

$$\sigma_{\text{mean}} = \frac{\sigma_{\text{single}}}{\sqrt{N_{\text{reps}}}}. \tag{10}$$

This averaging is appropriate when only the average response is needed; it cannot be applied to SSRO since averaging destroys the per-shot state information.

The practical workflow is: (1) use `acquire_decimated()` to find the correct `adc_trig_offset` and verify signal quality; (2) switch to `acquire()` (accumulated) for all data-taking, using single shots for SSRO and repeat-averaged shots for spectroscopy and calibration experiments.

5.3 Resonator Spectroscopy

The resonator bare frequency ω_r and linewidth κ are found by sweeping the probe frequency around the estimated resonator frequency and measuring the transmitted or reflected IQ amplitude. In QICK, there are two sweep strategies:

Fast sweep (`RAveragerProgram`). The tProcessor increments the DAC DDS frequency register at each experiment step via the `update()` method. This approach is fast because the FPGA handles all steps with no Python round-trips. However, the ADC NCO frequency is set via the ARM AXI bus and cannot be updated by the tProcessor. For resonator spectroscopy, both the DAC and ADC frequencies must track together; this is not achievable with the fast method.

Slow sweep (Python outer loop). A `for` loop in Python updates both the DAC generator frequency and the ADC readout frequency at each step. The per-step overhead is a few milliseconds (dominated by network round-trips in remote mode), but this approach can sweep *any* parameter, including the ADC NCO. This is the correct approach for resonator spectroscopy.

In practice, a Python loop steps both the generator and readout frequencies together, runs an `AveragerProgram` at each point, and records the magnitude $|I + jQ|$ of the accumulated phasor. The resonator frequency is identified as the dip (reflection) or peak (transmission) in the swept amplitude response.

5.4 Multiplexed Readout

For multi-qubit systems, simultaneous readout of several qubits on the same physical line requires *multiplexed readout*: a broadband probe signal containing multiple tones at the individual resonator frequencies $\{f_{r,k}\}$ [11, 12]. The QICK framework supports this through a *Polyphase Filter Bank* (PFB) readout mode.

A PFB is a bank of digital bandpass filters implemented as a single, efficient DSP operation (polyphase decomposition of a prototype filter followed by an FFT). It takes the broadband ADC stream and simultaneously demodulates N equally-spaced frequency channels, analogous to a digital spectrum analyser. Advantages over the single-tone DDS approach include:

- No need to retune the ADC NCO between qubits.
- Computational cost scales as $O(N \log N)$ rather than $O(N)$ independent demodulators.
- Sidesteps the ADC-frequency-sweep limitation: each PFB output channel is a fixed digital filter, not a tunable DDS.

Multiplexed readout via the QICK PFB firmware was not implemented within the scope of this work. The single-tone DDS architecture described in Sections 5.2 and 5.3 was used throughout. A natural next step is to extend the architecture to cover the four Dátil resonator frequencies characterised at Qilimanjaro: 6.505, 6.730, 6.978, and 7.389 GHz. These frequencies lie above the 6 GHz upper edge of the XM655 BALUN passband, so simultaneous multiplexed readout at all four tones would require direct HD-header connections with discrete BALUNs, as discussed in Section 3. The PFB channel spacing and prototype filter bandwidth would also need to be verified against the 225 MHz minimum separation between adjacent resonators in this device.

6 Real-Time Feedback and Mid-Circuit Measurement

A key motivation for FPGA-based control is the ability to execute *real-time* conditional logic: perform a measurement, make a decision, and apply a corrective pulse, all on the FPGA, without a round-trip to the host CPU. This section describes the tProcessor V2 instruction set, demonstrates the conditional branching primitive, and outlines the active qubit reset protocol.

6.1 tProcessor Architecture

The tProcessor V2 is a soft-processor synthesised in the FPGA fabric. It executes a custom instruction set tailored to pulse generation and readout sequencing. Key features include:

- **Register file**: 32 general-purpose 64-bit registers per channel page, plus cross-page registers for inter-channel synchronisation.
- **Pulse triggers**: dedicated instructions that fire DAC signal generators or arm ADC acquisition windows.
- **Timing instructions**: `synci` (advance tProcessor time without touching channel timelines), `sync_all` (synchronise all channels), `wait_all` (stall until all readout windows close).
- **Conditional branch**: `condj page, reg_a, op, reg_b, label` jumps to `label` if `reg_a op reg_b` is true.

All branching targets are resolved at *compile time* by the Python assembler: Python's sequential execution of `body()` builds an instruction list, `label()` records the current position as a named address, and the assembler backpatches jump targets before loading the program onto the tProcessor. By the time the program runs, only integer addresses exist; there is no runtime symbol table.

6.2 Conditional Logic Demonstration

The QICK Demo 3 notebook demonstrates the core conditional-branch primitive:

```
def body(self):
    cfg = self.cfg
    self.trigger(adcs=[cfg["ro_ch"]],
                 adc_trig_offset=cfg["adc_trig_offset"])
    # condj: jump to SKIP_PULSE if number < threshold
    self.condj(0, 2, '<', 1, 'SKIP_PULSE')
    self.pulse(ch=cfg["res_ch"])          # skipped if condition true
    self.label('SKIP_PULSE')
    self.wait_all()
    self.sync_all(self.us2cycles(cfg["relax_delay"]))
```

Registers 1 and 2 are pre-loaded with the threshold and the test value via `regwi`. When `number=100, threshold=50`: the condition `100 < 50` is false, the pulse fires, and the ADC captures a loopback signal. When `number=10, threshold=50`: the condition is true, the pulse is skipped, and the ADC trace is flat noise.

Timing subtlety. Inside a `body()` with an open ADC acquisition window, `synci` must be used to advance time between pulses, not `sync_all`. `sync_all` advances all channel timelines to the furthest point, which is the end of the ADC window; subsequent pulses would then be scheduled after the window has closed. `synci` advances only the tProcessor's internal counter, leaving the ADC timeline undisturbed.

6.3 π Pulse Calibration

A π pulse rotates the qubit state by 180° on the Bloch sphere: $|0\rangle \rightarrow |1\rangle$ and $|1\rangle \rightarrow |0\rangle$. It is a fundamental primitive for qubit initialisation (active reset) and single-qubit gates. Calibrating it requires a connected qubit chip; this section describes the procedure that would be followed once the ZCU216 is connected to a device.

The π pulse amplitude is found via a *Rabi experiment*: the qubit drive pulse amplitude is swept from zero, and the readout signal oscillates between the $|0\rangle$ and $|1\rangle$ IQ clusters as the qubit is driven through successive half-turns on the Bloch sphere. The first amplitude at which the signal crosses from the $|0\rangle$ cluster to the $|1\rangle$ cluster determines the π pulse amplitude. The Rabi sequence is:

1. Prepare qubit in $|0\rangle$ (wait $\gg T_1$).
2. Apply qubit drive pulse at frequency ω_{01} with variable amplitude A and fixed duration.
3. Read out via the resonator.

The qubit drive tone is at ω_{01} (the qubit transition frequency, typically 5 GHz to 7 GHz for fluxonium), which is distinct from the resonator frequency ω_r used for readout. Two separate generator channels are therefore required: one for readout, one for qubit control. The QICK firmware supports this natively: the tProcessor can address both channels independently with nanosecond timing precision.

6.4 Active Qubit Reset

Active reset rapidly returns the qubit to $|0\rangle$ by measuring its state and applying a corrective π pulse only if the qubit is found in $|1\rangle$ [13]. The QICK tProcessor V2 has all the primitives required to implement this entirely on-FPGA: the conditional branch demonstrated in Section 6.2 provides the decision logic, and the two-channel architecture supports simultaneous readout and drive. The following protocol was designed for deployment once the board is connected to a qubit chip:

1. **Dispersive readout**: fire the readout tone and acquire the accumulated IQ result.
2. **Thresholding**: the readout firmware compares the IQ result against a pre-calibrated threshold and writes a binary decision (0 or 1) into a tProcessor register.
3. **Conditional π pulse**: `condj` checks the decision register. If the qubit is in $|1\rangle$, a calibrated π pulse is applied on the qubit drive channel. Otherwise, the pulse is skipped.
4. **Wait**: allow any residual photons in the resonator to decay before starting the experiment.

Because the entire decision-to-pulse path would run inside the tProcessor, the latency between completing the readout window and firing the corrective pulse is determined solely by FPGA clock cycles. For the ZCU216 operating at a 384 MHz fabric clock, this latency is of order tens of nanoseconds, orders of magnitude faster than a software round-trip.

6.5 Latency Budget

Table 3 gives an estimated latency breakdown for the active reset sequence.

Table 3: Estimated latency contributions in the active reset loop.

Stage	Latency	Notes
Readout pulse duration	0.5 µs to 2 µs	Depends on κ
ADC integration window	0.5 µs to 2 µs	
FPGA decision (threshold)	∼10 ns	tProcessor cycles
Conditional branch	∼3 ns	1 tProc cycle
π pulse duration	50 ns to 500 ns	Depends on T_1
Total (excl. readout)	<1 µs	

The dominant latency is the readout integration window; the digital decision and conditional fire together contribute less than 100 ns, well within qubit coherence times of 10 µs to 100 µs for state-of-the-art fluxonium devices.

7 ML-Assisted State Discrimination

7.1 IQ Data and the Discrimination Problem

Each single-shot readout produces an averaged (I, Q) pair, a point in the complex plane. Repeating the preparation and measurement builds up two clouds of such points, one for $|0\rangle$ and one for $|1\rangle$, whose centroids and spreads depend on the readout power, resonator parameters, and integration time. As set out in Section 2.5, for ideal Gaussian clusters

of equal covariance the optimal linear boundary is the perpendicular bisector of the line joining the centroids, which is precisely the LDA boundary (Eq. 5); QDA relaxes the equal-covariance assumption and gives a curved boundary. In practice, state-preparation errors, T_1 decay during the readout pulse, and higher-level leakage ($|2\rangle$, $|3\rangle$) distort the clusters from ideal Gaussians and motivate more expressive classifiers. Throughout, we quote the assignment fidelity

$$\mathcal{F} = \tfrac{1}{2}\big[P(\hat{s}=0 \mid s=0) + P(\hat{s}=1 \mid s=1)\big], \tag{11}$$

the average of the two per-class accuracies.

7.2 Lightweight Classifiers for FPGA Deployment

For eventual on-FPGA deployment, the classifier must be computable using only fixed-point arithmetic and a small number of multiply-accumulate operations. Three constraints matter: latency (the decision must complete within one tProcessor cycle budget, $\lesssim$10 ns, so as not to extend the readout dead time), resource usage (it must fit in the available DSP slices and block RAM without displacing the readout and tProcessor logic), and trainability (it must train on a laptop from a few thousand calibration shots).

With these constraints in mind, three classifier families are considered. A **linear threshold** (LDA) reduces to a single dot product and a comparison, the minimum possible arithmetic, directly implementable in the tProcessor's `mathi` instruction. A **quadratic threshold** (QDA) evaluates a 2×2 matrix product (4 multiply-accumulates) and handles asymmetric cluster covariances. A **small MLP** with inputs (I, Q), one or two hidden layers of 4–64 neurons with ReLU activations, and a sigmoid output can be computed as a matrix-vector product and fits in $\lesssim$ 200 18-bit DSP slices on the ZCU216. The following sections report results from evaluating these classifiers offline on existing experimental data; FPGA deployment is discussed in Section 7.7. For prior work on neural-network-based discrimination, including multiplexed multi-qubit readout, see Lienhard et al. [14].

7.3 Experimental Data

The dataset used throughout this chapter is `SSRO.h5`, collected on a superconducting qubit device at Qilimanjaro. It contains 2000 single-shot IQ measurements per prepared state ($|0\rangle$ and $|1\rangle$), accumulated on-FPGA by the rectangular integrator in `axis_readout_v2`. Each shot is a single (I, Q) pair, the time-averaged quadrature amplitudes over the readout window. The class centroids and separations extracted from this dataset are summarised in Table 4.

Table 4: IQ blob statistics extracted from `SSRO.h5`.

Quantity	$\|0\rangle$	$\|1\rangle$
Centroid μ_I (ADU)	-0.32	-2.10
Centroid μ_Q (ADU)	$+4.50$	$+6.31$
IQ separation (ADU)	2.53	
IQ correlation	-0.587	-0.664

The negative IQ correlation in both states indicates a rotated noise ellipse, consistent with a readout tone that is not perfectly aligned to the I axis. The data were split 80/20 into training and test sets (3200 / 800 shots) using stratified sampling.

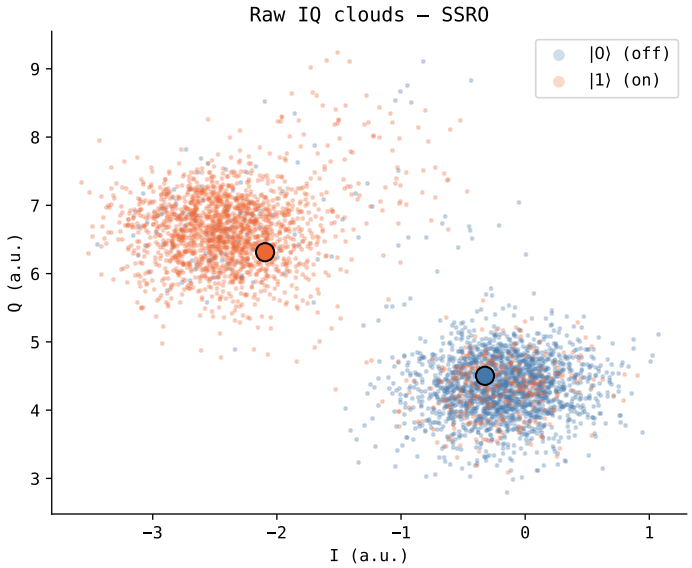

Figure 2: IQ scatter plot of the `SSRO.h5` dataset (2000 shots per state). Blue: $|0\rangle$ (qubit off); orange: $|1\rangle$ (qubit on). Filled circles mark the cluster centroids. The two clouds are well-separated and approximately Gaussian, with the $|1\rangle$ ellipse visibly wider, consistent with the $\approx 1.7\times$ covariance asymmetry in Table 4. The diagonal orientation of both ellipses reflects the residual IQ rotation noted in the text.

7.4 Part I: Integrated IQ Discrimination

LDA, QDA, and a representative MLP (hidden layers 32→16, ReLU) were trained on the integrated IQ data. Table 5 reports the assignment fidelity on the held-out test set.

Table 5: Assignment fidelity on integrated IQ data (test set, 800 shots).

Classifier	$\mathcal{F}$	$\|0\rangle$ acc.	$\|1\rangle$ acc.
LDA	0.906	0.940	0.873
QDA	0.906	0.940	0.873
MLP (32→16)	0.906	0.940	0.873

All three classifiers achieve identical fidelity. The QDA result is the informative one: the two IQ clouds do *not* share the same covariance (the $|1\rangle$ covariance is roughly $1.7\times$ larger than the $|0\rangle$ covariance, Table 4), so QDA's curved boundary is in principle more expressive than LDA's linear one. That it recovers no additional fidelity is explained by the large cluster separation: the Mahalanobis distance between the centroids is approximately 2.4, which places the clusters well into the regime where the optimal boundary is nearly linear regardless of covariance asymmetry. LDA, QDA, and the MLP are all saturating the classification capacity of the two-dimensional integrated IQ input.

The $|0\rangle$ accuracy (94.0%) exceeds the $|1\rangle$ accuracy (87.3%), pointing at a systematic source of $|1\rangle$ misclassification. This asymmetry is the first diagnostic signature of T_1 relaxation during the readout window, which causes some $|1\rangle$ shots to produce an integrated IQ that falls in the $|0\rangle$ blob.

7.5 Part II: Architecture Search

To test whether the LDA–MLP parity in Part I is specific to the (32→16) architecture, a systematic grid search was performed over 20 MLP configurations spanning 1–3 hidden layers and 4–64 neurons per layer (17 to 6465 parameters), evaluated by 5-fold stratified cross-validation.

Table 6: Representative architecture grid search results (5-fold CV fidelity); see Figure 3 for all 20 configurations. LDA baseline: 0.898 ± 0.013. No MLP architecture exceeds LDA.

Architecture	$\mathcal{F}$ (CV)	$\pm$ std	Δ vs. LDA
(4,)	0.897	0.012	−0.001
(64,)	0.897	0.012	−0.001
(8, 8)	0.896	0.012	−0.002
(32, 16)	0.897	0.012	−0.001
(64, 64)	0.897	0.012	−0.001
(64, 64, 32)	0.897	0.012	−0.002

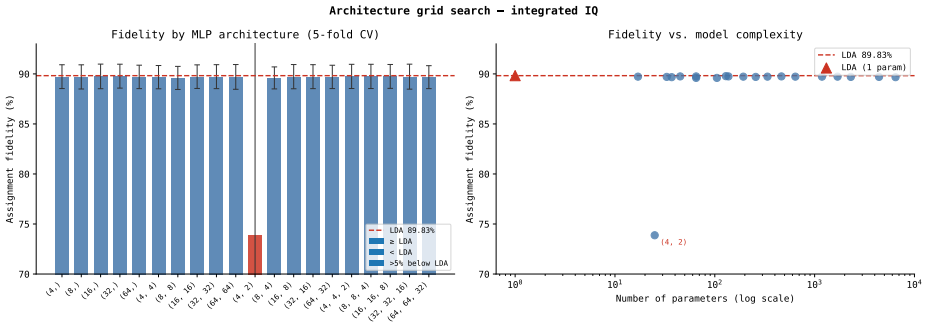

Figure 3: Architecture grid search results on integrated IQ data (5-fold CV). *Left*: assignment fidelity per MLP architecture; the red dashed line is the LDA baseline (89.83%). All architectures except the degenerate (4, 2) configuration lie within CV noise of LDA. *Right*: fidelity vs. parameter count on a log scale. No trend with model size is visible across three orders of magnitude. The (4, 2) outlier at $\approx$74% reflects under-parameterisation rather than overfitting, and is excluded from Table 6.

The fidelity landscape is flat: no architecture, across three orders of magnitude in parameter count, improves on LDA. This is not a failure of the models but a statement about the data, and it follows directly from Part I. The clusters are well-separated (Mahalanobis distance ≈ 2.4), so the optimal boundary is nearly linear even given the $\approx$1.7× covariance asymmetry, as QDA matching LDA confirms. LDA already finds a near-Bayes-optimal boundary, and no classifier can improve on it from the same two-dimensional integrated IQ features.

Further improvement must therefore come from richer inputs, not a more expressive classifier. The $|1\rangle$ accuracy deficit diagnosed in Part I points to the mechanism: T_1 relaxation mid-shot.

7.6 Part III: Raw-Trace CNN

7.6.1 Motivation and the T1 Relaxation Problem

As established in Section 2.6, no classifier operating on integrated IQ can recover a shot in which T_1 relaxation occurs mid-window: the integrator averages over both signal levels and

the point lands between the two clusters, destroying the information about the prepared state. The raw ADC trace, by contrast, encodes the relaxation time t^* directly as a step in the IQ trajectory, and a 1D CNN can detect this step and recover the correct label. Convolutional filters are suited to exactly this kind of local temporal structure, which pointwise classifiers cannot exploit.

Part III demonstrates the capability on physically-grounded synthetic traces, since raw decimated traces from the ZCU216 are not yet available: acquiring them requires `acquire_decimated` rather than the standard `acquire` call, and either a dedicated `SSRO_raw_synthetic.h5` dataset (the simulated counterpart used here) or real hardware traces with the board connected to a qubit chip.

7.6.2 Synthetic Trace Model

Each shot is modelled as a noisy cavity response:

$$\begin{pmatrix} i(t) \\ q(t) \end{pmatrix} = \frac{e(t)}{\bar{e}} \boldsymbol{\mu}_{s(t)} + \boldsymbol{\eta}(t), \qquad e(t) = 1 - e^{-t/\kappa}, \tag{12}$$

where $\bar{e} = \langle e(t) \rangle_t$ normalises the envelope so that $\langle (i(t), q(t)) \rangle_t = \boldsymbol{\mu}_{s(t)}$, matching the real blob centroids exactly. The noise $\boldsymbol{\eta}(t) \sim \mathcal{N}(\mathbf{0}, T\Sigma_s)$ is chosen so that the sample mean has covariance Σ_s, matching the real blob widths.

For $|1\rangle$ shots, qubit relaxation is modelled by drawing a relaxation time t^* from a geometric distribution with rate $1 - e^{-1/T_1}$. After t^* the signal switches from $\boldsymbol{\mu}_1$ to $\boldsymbol{\mu}_0$:

$$\boldsymbol{\mu}_{s(t)} = \begin{cases} \boldsymbol{\mu}_1 & t < t^* \\ \boldsymbol{\mu}_0 & t \geq t^* \end{cases}. \tag{13}$$

Simulation calibration. The generator is anchored to the real data through its blob statistics: rectangular integration of the noiseless traces reproduces the measured `SSRO.h5` centroids and widths to within a few percent (the self-consistency check below). The relaxation time was set to $T_1 = 600$ samples ($\approx 3.0\,\mu s$ at 200 MSps), which gives a relaxation fraction of about 28% among $|1\rangle$ shots. This is deliberately larger than on the real device, where the measured $|1\rangle$ accuracy of 87.3% implies a much smaller relaxation population; the synthetic set over-represents mid-readout relaxation as a stress test of the recovery the CNN is meant to perform. Recalibrating T_1 to the real $|1\rangle$ error rate is left to future work (Section 8). The simulation parameters are summarised in Table 7.

Table 7: Synthetic trace simulation parameters. Blob centroids and widths match `SSRO.h5`; T_1 is set to over-represent mid-readout relaxation (see text).

Parameter	Physical meaning	Value
T	Readout window	200 samples ($\approx 1\,\mu s$)
$\tau_{\text{cav}} = \kappa^{-1}$	Cavity ringup time constant	20 samples ($\approx 100\,ns$)
T_1	Qubit relaxation time	600 samples ($\approx 3.0\,\mu s$)
Relaxation fraction	$\|1\rangle$ shots with $t^* < T$	$\approx$28%

As a self-consistency check, rectangular integration of the noiseless synthetic traces reproduces the `SSRO.h5` centroids to better than 1% (e.g. μ_Q of $|0\rangle$: +4.521 synthetic vs. +4.502 real) and the cluster widths to a few percent (e.g. σ_I of $|1\rangle$: 0.851 vs. 0.874).

7.6.3 CNN Architecture and Training

The `ReadoutCNN` model operates on raw traces of shape $(T, 2)$. Two `Conv1d` layers (channels $2 \to 8$ with kernel 8, stride 2, then $8 \to 16$ with kernel 4, stride 2) compress the time axis, global average pooling collapses it, and a two-layer dense head ($16 \to 16 \to 1$) produces the binary logit for $P(|1\rangle)$. The model has 953 parameters, sized for deployment via `hls4ml` [15]. It was trained for 60 epochs with binary cross-entropy loss and the Adam optimiser on 2000 shots per state, split 80/20 into training and test sets. The training loss converges smoothly within about 40 epochs and the test fidelity plateaus near 99.75%, with no sign of overfitting, consistent with the small model size.

7.6.4 Results

Table 8 compares LDA on integrated IQ against the CNN on raw traces, evaluated on the same held-out test set (20%, 800 shots).

Table 8: CNN vs. LDA on synthetic raw traces. LDA operates on the time-averaged (I, Q); the CNN operates on the full trace $(T, 2)$. The synthetic set over-represents relaxation relative to the real device (see text), so the LDA fidelity here is lower than the 0.906 of Table 5. All values from the held-out test set ($n = 800$; 122 relaxation shots).

Classifier	$\mathcal{F}$	$\|0\rangle$ acc.	$\|1\rangle$ acc. (all)	$\|1\rangle$ acc. (relax. only)
LDA (integrated IQ)	0.826	0.875	0.778	0.443
1D CNN (raw trace)	0.9975	1.000	0.995	0.984
CNN gain	+17.1%	+12.5%	+21.8%	+54.1%

The CNN classifies the 122 relaxation shots in the held-out test set with 98.4% accuracy, against 44.3% for LDA, a gain of 54.1 percentage points on the shots that are essentially invisible to integration-based methods. This confirms the expectation from Section 2.6: the CNN exploits the temporal jump structure of the raw trace rather than merely interpolating between blob positions. The $P(|1\rangle)$ confidence rises monotonically with relaxation time t^*: shots that relax late in the window (spending most of the trace in $|1\rangle$) are classified with near-certainty, while shots that relax early (spending most of the trace in $|0\rangle$) are correctly flagged as ambiguous (Appendix B, Figure 6).

These results are simulation estimates anchored to the real device data. As noted above, the synthetic dataset uses $T_1 = 600$ samples ($\approx$3.0 µs at 200 MSps), giving a relaxation fraction of about 28% (122 relaxation events in the test set of 800 shots), well above the real device. This inflates the absolute gap between CNN and LDA, so the headline 17-point fidelity improvement should not be read as the gain expected on real data. What matters is the qualitative conclusion, and it is robust: a CNN operating on raw traces has access to temporal information that the rectangular integrator discards, and this structural advantage persists regardless of the exact T_1. Retraining on real raw traces, once collected with `acquire_decimated`, is the natural next step (Section 8).

7.7 FPGA Deployment Path

FPGA deployment of the trained classifiers was not achieved within the scope of this work. This section outlines the concrete steps required to close the loop from the trained models to real-time discrimination inside the QICK firmware.

The current readout pipeline (a rectangular integrator followed by the tProcessor threshold register) already implements LDA implicitly. Formalising this as an explicit

dot-product instruction (`mathi`) adds no hardware overhead. The CNN requires a separate data path: raw decimated traces from `acquire_decimated` are fed into an `hls4ml`-synthesised IP block that produces a binary decision within approximately 100 ns to 200 ns at 250 MHz, well within the tProcessor's feedback budget (Table 3).

The full deployment sequence is:

1. **Collect real raw traces**: connect the ZCU216 to a qubit chip and acquire `SSRO_raw.h5` with `acquire_decimated` (shape $(2, N_{\text{shots}}, T, 2)$), setting `readout_length` to at least $3/\kappa$ to capture the full cavity ringdown.
2. **Retrain and validate**: retrain on real traces, calibrating the synthetic T_1 to the measured $|1\rangle$ error rate so the synthetic set can supplement the real data.
3. **Export to HLS**: convert the trained model (PyTorch $\rightarrow$ ONNX) with `hls4ml` [15], targeting the ZCU216 fabric (`xczu49dr`), `io_stream`, and `ap_fixed<16,6>` precision.
4. **Synthesise and integrate**: build the HLS project in Vivado and add the IP block to the QICK bitstream alongside the existing readout chain.
5. **Benchmark**: compare LDA-threshold, CNN-IP, and host-side discrimination on the same data to quantify the latency and fidelity gains.

Based on the model's 953 parameters and typical `hls4ml` resource scaling for comparable architectures, the CNN IP block is expected to require on the order of 10 000 LUTs and tens of DSP slices, a small fraction of the ZCU216's available fabric, though this must be confirmed by synthesis.

8 Conclusions and Outlook

8.1 Summary

This thesis has documented the setup, characterisation, and initial evaluation of a QICK-based readout architecture on a ZCU216 RFSoC, together with the design of a machine-learning-assisted discrimination pipeline on existing experimental data. The main contributions are:

1. **System integration**: The ZCU216 board, XM655 analogue front-end, and QICK firmware were brought up from scratch, including SSH access, PYNQ configuration, and a persistent Pyro4 remote-operation server that preserves calibration state across notebook sessions.
2. **Phase-coherent readout calibration**: A procedure was developed to measure the random DAC–ADC DDS phase offset at each power-on and fit the linear model $\phi(f) = \phi_0 - 360^\circ \tau f$, with the fitted `res_phase` folded into the standard experiment configuration.
3. **Readout data-path**: The decimated and accumulated readout modes were characterised. Accumulated readout gives one integrated (I, Q) pair per shot for SSRO; repeat-averaging improves spectroscopy SNR but destroys the single-shot information. A resonator spectroscopy workflow was established for single-qubit and multiplexed scenarios.

4. **Real-time feedback primitives**: The tProcessor V2 conditional branch (`condj`) was demonstrated in loopback, and an active-reset sequence (dispersive readout, thresholding, calibrated π pulse) was designed with an estimated digital decision-to-fire latency below 100 ns.

5. **ML-assisted discrimination**: On integrated IQ data, LDA reaches an assignment fidelity of 90.6% ($|0\rangle$: 94.0%, $|1\rangle$: 87.3%), and a grid search over 20 MLP architectures finds no nonlinear classifier that improves on it, as expected for near-Gaussian, linearly separable blobs. The $|1\rangle$ deficit is the signature of T_1 relaxation mid-shot. A 953-parameter 1D CNN on synthetic raw traces reaches 99.75% overall fidelity and 98.4% accuracy on the 122 relaxation shots, against 44.3% for LDA; because the synthetic set over-represents relaxation, the robust conclusion is the qualitative advantage of raw-trace input rather than the size of the gap. FPGA deployment was not achieved within the scope of this work; Section 7.7 outlines the path toward it.

8.2 Outlook

Several extensions follow directly from the results above. The first experimental priority is to collect a real `SSRO_raw.h5` dataset from the ZCU216 with `acquire_decimated`, retrain the CNN, and check that the T_1-relaxation recovery generalises to real data; the synthetic T_1 should first be recalibrated to the measured $|1\rangle$ error rate ($\approx 12.75\%$, implying $T_1 \approx 1200$–1500 samples rather than the 600 used here). With real traces in hand, the CNN can be exported via `hls4ml` and synthesised as a fixed-point IP block in the QICK bitstream (Section 7.7), closing the active-reset loop at full speed without a software round-trip.

Beyond a single qubit, extending to multiplexed readout via the QICK PFB firmware would allow simultaneous readout of all qubits in Qilimanjaro's multi-qubit chips, a prerequisite for quantum error-correction cycles. The QICK-based readout layer should also be integrated with Qilimanjaro's higher-level programming interfaces (circuit compilation, calibration databases, experiment management) for a production-ready stack. Finally, the fluxonium dispersive-shift landscape is two-dimensional in the (Φ_z, Φ_x) flux space, and an ML-assisted optimisation of the readout circuit parameters (coupling capacitance, resonator frequency) to maximise the cavity pull over a large flux region is a complementary direction being pursued in a related BSc project within the team.

Bibliography

[1] Leandro Stefanazzi et al. QICK: Quantum instrumentation control kit for superconducting quantum computers. *Review of Scientific Instruments*, 93(4):044709, 2022. arXiv:2110.00557.

[2] Philip Krantz, Morten Kjaergaard, Fei Yan, Terry P. Orlando, Simon Gustavsson, and William D. Oliver. A quantum engineer's guide to superconducting qubits. *Applied Physics Reviews*, 6(2):021318, 2019.

[3] Jens Koch, Terri M. Yu, Jay Gambetta, A. A. Houck, D. I. Schuster, J. Majer, Alexandre Blais, M. H. Devoret, S. M. Girvin, and R. J. Schoelkopf. Charge-insensitive qubit design derived from the Cooper pair box. *Physical Review A*, 76(4):042319, 2007.

[4] Vladimir E. Manucharyan, Jens Koch, Leonid I. Glazman, and Michel H. Devoret. Fluxonium: Single Cooper-pair circuit free of charge offsets. *Science*, 326(5949):113–116, 2009.

[5] Alexandre Blais, Ren-Shou Huang, Andreas Wallraff, S. M. Girvin, and R. J. Schoelkopf. Cavity quantum electrodynamics for superconducting electrical circuits: An architecture for quantum computation. *Physical Review A*, 69(6):062320, 2004. arXiv:cond-mat/0402216.

[6] Alexandre Blais, Arne L. Grimsmo, S. M. Girvin, and Andreas Wallraff. Circuit quantum electrodynamics. *Reviews of Modern Physics*, 93(2):025005, 2021. arXiv:2005.12667.

[7] A. Wallraff, D. I. Schuster, A. Blais, L. Frunzio, R.-S. Huang, J. Majer, S. Kumar, S. M. Girvin, and R. J. Schoelkopf. Strong coupling of a single photon to a superconducting qubit using circuit quantum electrodynamics. *Nature*, 431(7005):162–167, 2004.

[8] Jay Gambetta, Alexandre Blais, Maxime Boissonneault, Andrew A. Houck, D. I. Schuster, and S. M. Girvin. Protocols for optimal readout of qubits using a continuous quantum nondemolition measurement. *Physical Review A*, 76(1):012325, 2007.

[9] AMD / Xilinx. ZCU216 evaluation board user guide (UG1390). https://www.xilinx.com/products/boards-and-kits/zcu216.html, 2021.

[10] Fermilab Quantum Institute. QICK: Quantum instrumentation control kit. https://github.com/openquantumhardware/qick, 2022.

[11] Theodore Walter, Philipp Kurpiers, Simone Gasparinetti, Paul Magnard, Anton Potočnik, Yves Salathé, Marek Pechal, Mintu Mondal, Markus Oppliger, Christopher Eichler, and Andreas Wallraff. Rapid high-fidelity single-shot dispersive readout of superconducting qubits. *Physical Review Applied*, 7(5):054020, 2017.

[12] Johannes Heinsoo, Christian Kraglund Andersen, Ants Remm, Sebastian Krinner, Theodore Walter, Yves Salathé, Maiken Kalaee, Simon J. M. Habraken, David P. DiVincenzo, Andreas Wallraff, and Christopher Eichler. Rapid high-fidelity multiplexed readout of superconducting qubits. *Physical Review Applied*, 10(3):034040, 2018.

[13] M. D. Reed, B. R. Johnson, A. A. Houck, L. DiCarlo, J. M. Chow, D. I. Schuster, L. Frunzio, and R. J. Schoelkopf. Fast reset and suppressing spontaneous emission of a superconducting qubit. *Applied Physics Letters*, 96(20):203110, 2010.

[14] Benjamin Lienhard, Antti Vepsäläinen, Luke C. G. Govia, Cole R. Hoffer, Jack Y. Qiu, Diego Ristè, Matthew Ware, David Kim, Roni Winik, Alexander Melville, Bethany Niedzielski, Jonilyn Yoder, Guilhem J. Ribeill, Thomas A. Ohki, Hari K. Krovi, Terry P. Orlando, Simon Gustavsson, and William D. Oliver. Deep-neural-network discrimination of multiplexed superconducting-qubit states. *Physical Review Applied*, 17(1):014024, 2022. arXiv:2102.12481.

[15] Farah Fahim, Benjamin Hawks, Christian Herwig, James Hirschauer, Sergo Jindariani,

Nhan Tran, Luca P. Carloni, Giuseppe Di Guglielmo, Philip Harris, Jennifer Hasler, Duc Hoang, Shih-Chieh Huang, Vladimir Loncar, Yichen Luo, Jennifer Ngadiuba, Maurizio Pierini, Dylan Rankin, Sioni Summers, and Jian Weng. hls4ml: An open-source codesign workflow to empower scientific low-power machine learning devices. *arXiv preprint*, 2021. arXiv:2103.05579.

A Key Code Listings

This appendix collects the most important Python/QICK code snippets used in this thesis. Full notebooks are available in the accompanying GitHub repository.

A.1 Phase Calibration Programme

```python
class PhaseCalProgram(AveragerProgram):
    """Sweeps frequency and measures DAC-ADC phase offset."""

    def initialize(self):
        cfg = self.cfg
        self.declare_gen(ch=cfg["res_ch"], nqz=1)
        self.declare_readout(ch=cfg["ro_ch"],
                             length=cfg["readout_length"],
                             freq=cfg["pulse_freq"],
                             gen_ch=cfg["res_ch"])
        freq_reg = self.freq2reg(cfg["pulse_freq"],
                                 gen_ch=cfg["res_ch"],
                                 ro_ch=cfg["ro_ch"])
        self.set_pulse_registers(ch=cfg["res_ch"],
                                 style="const",
                                 freq=freq_reg,
                                 phase=0,
                                 gain=cfg["pulse_gain"],
                                 length=cfg["length"])
        self.synci(200)

    def body(self):
        self.measure(pulse_ch=self.cfg["res_ch"],
                     adcs=[self.cfg["ro_ch"]],
                     adc_trig_offset=self.cfg["adc_trig_offset"],
                     wait=True,
                     syncdelay=self.us2cycles(self.cfg["relax_delay"]))
```

A.2 Fast Amplitude Sweep with `RAveragerProgram`

```python
class SweepProgram(RAveragerProgram):
    """Sweeps DAC gain inside the FPGA without Python overhead."""

    def initialize(self):
        cfg = self.cfg
        self.declare_gen(ch=cfg["res_ch"], nqz=1)
        self.declare_readout(ch=cfg["ro_ch"],
                             length=cfg["readout_length"],
                             freq=cfg["pulse_freq"],
                             gen_ch=cfg["res_ch"])
        freq_reg = self.freq2reg(cfg["pulse_freq"],
                                 gen_ch=cfg["res_ch"],
                                 ro_ch=cfg["ro_ch"])
        self.set_pulse_registers(ch=cfg["res_ch"],
                                 style="const",
                                 freq=freq_reg,
                                 phase=cfg["res_phase"],
                                 gain=cfg["start"],
                                 length=cfg["length"])
```

```
        self.r_rp   = self.ch_page(cfg["res_ch"])
        self.r_gain = self.sreg(cfg["res_ch"], "gain")
        self.synci(200)

    def body(self):
        self.measure(pulse_ch=self.cfg["res_ch"],
                     adcs=[self.cfg["ro_ch"]],
                     adc_trig_offset=self.cfg["adc_trig_offset"],
                     wait=True,
                     syncdelay=self.us2cycles(self.cfg["relax_delay"]))

    def update(self):
        self.mathi(self.r_rp, self.r_gain, self.r_gain,
                   '+', self.cfg["step"])
```

A.3 Active Reset Sequence (Pseudocode)

Algorithm 1 Active qubit reset

Require: Calibrated π-pulse amplitude A_π and readout threshold θ

Trigger ADC readout window
Play readout tone on resonator channel
wait for readout window to close
$b \leftarrow \text{FPGA\_threshold}(I + jQ, \theta)$ ▷ $b = 1$ if qubit in $|1\rangle$
regwi $r_b \leftarrow b$
condj $r_b < 1$ **goto** SKIP_PI
Play π pulse on qubit drive channel
label SKIP_PI
synci t_{relax} ▷ Allow cavity photons to decay

B Synthetic-Trace Model and CNN

This appendix collects the core code behind the raw-trace study of Section 7.6 together with the figures it produces. The synthetic generator and the `ReadoutCNN` are reproduced below in condensed form, omitting data loading, calibration of the per-sample noise covariances, and plotting; the full `SSRO_synthetic_CNN.ipynb` notebook is in the same repository as the listings of Appendix A.

B.1 Synthetic Trace Generator

Each shot is a noisy cavity response sampled over T points. For $|1\rangle$ shots, the signal switches from $\boldsymbol{\mu}_1$ to $\boldsymbol{\mu}_0$ at a relaxation sample t^* drawn from a geometric distribution with per-sample rate $1 - e^{-1/T_1}$ (the discrete-time analogue of exponential T_1 decay).

```
def generate_traces(state, N, T, signal0, signal1, cov0_ps, cov1_ps,
                    T1_samples, rng):
    """Generate N synthetic raw IQ traces (N, T, 2) for a prepared state.
    |1> shots may relax at sample t* ~ Geometric(1 - exp(-1/T1))."""
    L0 = np.linalg.cholesky(cov0_ps)
    L1 = np.linalg.cholesky(cov1_ps)
    z  = rng.standard_normal((N, T, 2)).astype(np.float32)
```

```
    relax_times = np.full(N, -1, dtype=int)

    if state == 0:                          # |0>: constant state, no relaxation
        traces = signal0[None, :, :] + z @ L0.T.astype(np.float32)
    else:                                   # |1>: may decay to |0> at t*
        p = 1.0 - np.exp(-1.0 / T1_samples)
        u = rng.uniform(0, 1, N)
        t_star = np.floor(np.log(u) / np.log(1 - p)).astype(int)
        relaxes = t_star < T
        relax_times[relaxes] = t_star[relaxes]
        sig = np.tile(signal1[None, :, :], (N, 1, 1)).copy()
        for i in np.where(relaxes)[0]:
            sig[i, t_star[i]:, :] = signal0[t_star[i]:, :]
        traces = sig + z @ L1.T.astype(np.float32)
    return traces, relax_times
```

B.2 ReadoutCNN and Training

```
class ReadoutCNN(nn.Module):
    """1D CNN on raw IQ traces. Input (batch, T, 2) -> logit for P(|1>).
    953 params; hls4ml/ZCU216 target, ap_fixed<16,6>, ~100-200 ns @ 250 MHz."""
    def __init__(self):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv1d(2, 8,  kernel_size=8, stride=2, padding=3), nn.ReLU(),
            nn.Conv1d(8, 16, kernel_size=4, stride=2, padding=1), nn.ReLU(),
        )
        self.head = nn.Sequential(
            nn.AdaptiveAvgPool1d(1), nn.Flatten(),
            nn.Linear(16, 16), nn.ReLU(), nn.Linear(16, 1),
        )
    def forward(self, x):                   # (batch, T, 2) -> (batch, 2, T)
        return self.head(self.conv(x.permute(0, 2, 1)))

model     = ReadoutCNN().to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=EPOCHS)
loss_fn   = nn.BCEWithLogitsLoss()          # EPOCHS = 60, batch size 256

for epoch in range(EPOCHS):
    model.train()
    for xb, yb in dl_train:
        xb, yb = xb.to(device), yb.to(device)
        loss = loss_fn(model(xb), yb)
        optimizer.zero_grad(); loss.backward(); optimizer.step()
    scheduler.step()
```

B.3 Supplementary Figures

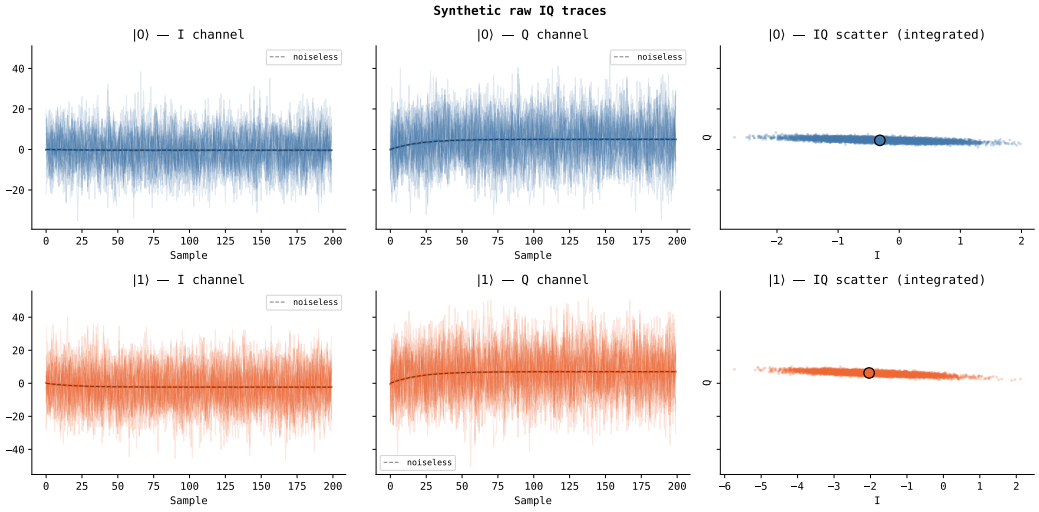

Figure 4: Synthetic raw IQ traces generated by Listing B.1's model and used in Section 7.6. Each panel overlays 200 noisy shots (faint) with the noiseless cavity-ringup envelope (dashed). *Top row*: $|0\rangle$ shots; both quadratures ring up and plateau near the $|0\rangle$ centroid. *Bottom row*: $|1\rangle$ shots; the Q channel plateaus at the $|1\rangle$ centroid. *Right column*: integrated IQ scatter after rectangular averaging; the $|1\rangle$ cloud is broader, consistent with the covariance asymmetry in Table 4. The traces are generated in an unrotated frame, so the IQ axes differ from Figure 2.

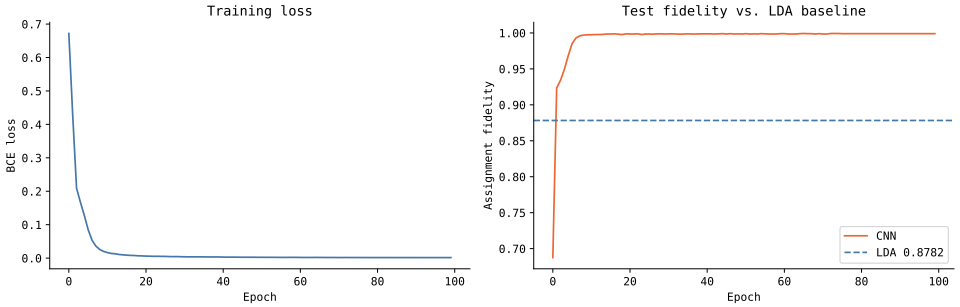

Figure 5: CNN training curves. *Left*: binary cross-entropy loss on the training set, converging smoothly within about 40 epochs. *Right*: test-set assignment fidelity per epoch, reaching $\approx 99.75\%$ and remaining well above the LDA baseline on the synthetic data (0.826, dashed). The absence of overfitting is consistent with the small model size (953 parameters).

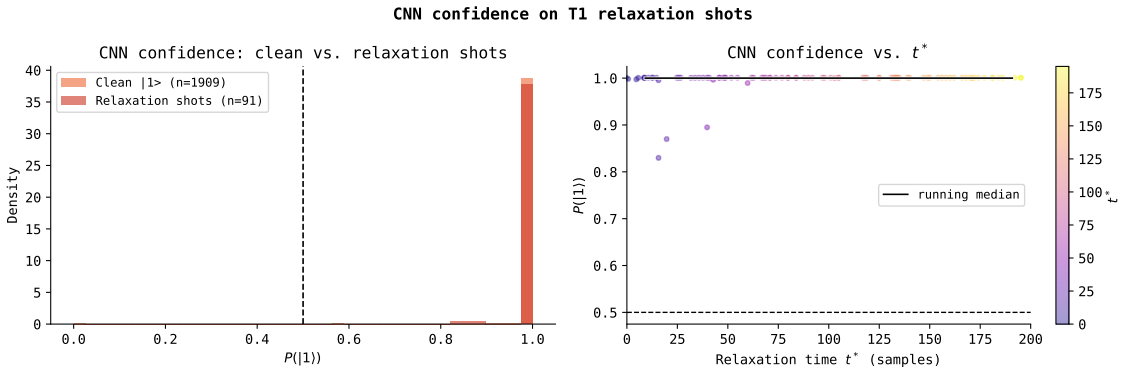

Figure 6: CNN confidence on T_1-relaxation shots, the mechanism behind the recovery reported in Section 7.6. *Left*: distribution of the CNN output $P(|1\rangle)$ for clean $|1\rangle$ shots (orange) versus shots that relaxed within the window (red); the relaxation shots spread toward the decision threshold. *Right*: $P(|1\rangle)$ versus relaxation time t^*, coloured by t^*, with a running median. Shots that relax late (most of the trace in $|1\rangle$) are classified with near-certainty, while shots that relax early are correctly driven toward ambiguity. LDA, which sees only the integrated mean, cannot make this distinction.